

UNIT -4

Pointers, Virtual Functions and Polymorphism: Introduction, Memory Management, new Operator and delete Operator, Pointers to Objects, this Pointer, Pointers to Derived Classes, Polymorphism, compile time polymorphism, Run time polymorphism, Virtual Functions, Pure Virtual Functions, Virtual Base Classes, Virtual Destructors, Function Overloading, Operator overloading, Rules for Operator overloading-binary and unary operators.

Introduction to Memory Management:

DYNAMIC MEMORY ALLOCATION & DEALLOCATION (new & delete)

C uses malloc() and calloc() functions to allocate memory dynamically at run time. It uses the function free() to deallocated dynamically allocated memory.

- C++ supports these functions, it defines two unary operators **new** and **delete** that perform the task of allocating and deallocating the memory in a better and easier way.
- A object can be created by using **new**, and destroyed by using **delete**.
- A data object created inside a block with **new**, will remain in existence until it is explicitly destroyed by using **delete**.

new operator:-

new operator can be used to create objects of any type .Hence new operator allocates sufficient memory to hold data of objects and it returns address of the allocated memory. Syntax:

```
pointer-variable = new data-type;
```

Ex: int *p = new int;

- To create memory space for arrays:

```
pointer-variable = new data-type[size];
```

Ex: int *p = new int[10];

delete operator:

If the variable or object is no longer required or needed is destroyed by “**delete**” operator, there by some amount of memory is released for future purpose. Syntax:

```
delete pointer-variable;
```

Eg: delete p;



If we want to free a dynamically allocated

array: delete [size] pointer-variable;

Program: write a program to find sum of list of integers

```
#include<iostream>
using
namespace std;
int main() {
int n,*p;
cout<<"Enter array
size:";
cin>>n; p=new
int[n];
cout<<"Enter list of integers"<<endl;
for(int i=0;i<n;i++) cin>>p[i];
//logic for summation int
s=0;
for( int i=0;i<n;i++) s=s+p[i];
cout<<"Sum of array elements is\n"; cout<<s;
delete [ ]p; return
0;
}
Enter array size:5
Enter list of integers
1 2 3 4 5
Sum of array elements is
15
```

Member Dereferencing operator: -

1.	Pointer to a member declarator	::*
2.	Pointer to member operator	->*
3.	Pointer to member operator	.*

Pointer to a member declarator ::*

This operator is used for declaring a pointer to the member of the class #include<iostream.h> class sample

```
{public:
int x;
};
int main()
{
sample s;    //object
int sample::*p;//pointer
declearation s.*p=10;    //correct
cout<<s.*p;
}
}
```

Output:10

2. Pointer to member operator -

```
>* #include<iostream.h> class sample
{ public:
    int x;
    void display()
    {
        cout<<"x="<<x<<endl;
    }
};

int main()
{
    sample s;    //object
    sample *ptr;
    int sample::*f=&sample::x;
    s.x=10; ptr=&s;
    cout<<ptr->*f;
    ptr->display();
}
```

3. Pointer to member operator

```
.* #include<iostream.h> class
sample
{ public: int x;
}; int
main()
{
    sample s;    //object
    int sample::*p;//pointer declaration
    s.*p=10;      //correct
    cout<<s.*p;
}
```

Pointers to Objects: Pointers to objects are useful for creating objects at run time. To access members



arrow operator () and de referencing operator or indirection (*) are used. **Declaration of pointer.**

className*ptr

ex:

item *obj;

Here obj is a pointer to object of type item.

class item

```
{ int code; float
  price;
```

public:

```
void getdata(int a,float b)
{ code=a;
  price=b;
}
```

```
void show()
```

```
{ cout<<"code:"<<code<<"\n"<<"Price:"<<price<<endl; }
```

```
};
```

Declaration of object and pointer to class item:

item obj; item
*ptr=&obj; The
member can be
accessed as
follow.

a) Accessing members using dot operator
obj.getdata(101,77.7); obj.show();

b) using pointer ptr->getdata(101,77.7);
ptr->show();

c) Using de referencing operator and dot operator
(*ptr).getdata(101,77.7);
(*ptr).show();

Creating array of objects using pointer:

```
item *ptr=new item[10];
```

Above declaration creates memory space for an array of 10 objects of type item.

```
#include<iostream.h>
class item
{ int code; float
  price;
public: void getdata(int a,float
  b)
  { code=a;
    price=b;
  } void
  show()
  { cout<<code<<"\t"<<price<<endl;
  } }; int main() { int n; int cd; float pri;
cout<<"Enter number of objects to be created:";
cin>>n; item *ptr=new item[n]; item *p;    p=ptr;
  for(int i=0;i<n;i++)
  { cout<<"Enter data for object"<<i+1;
    cout<<"\nEnter Code:";cin>>cd;
    cout<<"Enter price:";cin>>pri; p-
    >getdata(cd,pri); p++;
  }

  p=ptr;
  cout<<"Data in various objects are "<<endl; cout<<"Sno\tCode\tPrice\n";
  for(i=0;i<n;i++)
  { cout<<i+1<<"\t"; ptr-
    >show(); ptr++;
  }
return 0;
}
```

Pointers to Derived Classes: Pointers can be declared to derived class. it can be used to access members of base class and derived class. A base class pointer can also be used to point to object of derived class but it can access only members that are inherited from base class.

```
#include<iostream.h>
class base
{
public:
    int a;
    void get_a(int x)
    { a=x;
    }
    void display_a()
    { cout<<"In base"<<"\n"<<"a="<<a<<endl; } };
class derived:public base
{ int
b;
public:
    void get_ab(int x,int y)
    { a=x; b=y;
    }
    void display_ab()
    {
        cout<<"In Derived "<<"\n"<<"a="<<a<<"\nb="<<b<<endl;
    } };
int main() { base b; base *bptr;
bptr=&b;//points to the object of base class bptr-
>get_a(100); bptr->display_a();

    derived d; derived *dptr; dptr=&d;//points to the
    object of derived class

    dptr->get_a(400);
    dptr->display_a(); dptr->get_ab(300,200);
    dptr->display_ab();

    bptr=&d;//points to the object of derived
    class bptr->get_a(400); bptr->display_a();

return 0;
}
```

Output: In

base
a=100

In base
a=400
In Derived
a=300
b=200

In base
a=400

RUNTIME POLYMORPHISM USING VIRTUAL FUNCTIONS

Static & Dynamic Binding

Polymorphism means „one name “-„multiple forms.

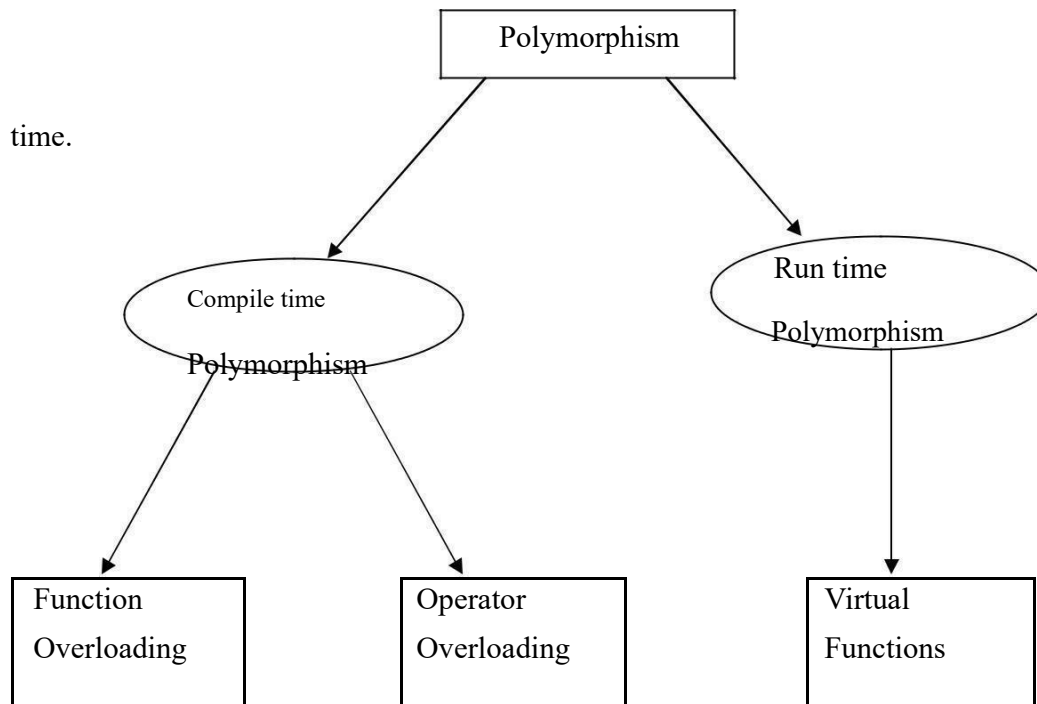
The overloaded member functions are „selected“ for invoking by matching arguments, both type and number. This information is known to the compiler at the compile time and compiler is able to select the appropriate function for a particular call at the compile time itself. This is called **Early Binding** or **Static Binding** or **Static Linking**. Also known as **compile time polymorphism**. Early binding means that an object is bound to its function call at the compile time.

It would be nice if the appropriate member function could be selected while the program is running. This is known as **runtime polymorphism**. C++ supports a mechanism known as **virtual function** to achieve run time polymorphism.

At the runtime, when it is known what class objects are under consideration, the appropriate version of the function is invoked. Since the function is linked with a particular class much later after the compilation, this process is termed as late binding. It is also known as **dynamic binding** because the

selection of the appropriate function is done dynamically at run

time.



VIRTUAL FUNCTIONS

Polymorphism refers to the property by which objects belonging to different classes are able to respond to the same message, but different forms. An essential requirement of polymorphism is therefore the ability to refer to objects without any regard to their classes.

When we use the same function name in both the base and derived classes, the function in the base class is declared as virtual using the keyword **virtual** preceding its normal declaration.

When a function is made virtual, C++ determines which function to use at runtime based on the type of object pointed to by the base pointer, rather than the type of the pointer. Thus, by making the base pointer to point to different objects, we can execute different versions of the virtual function.

```

#include<iostream.h>
    class Base
    { public:
    void display()
    {
    cout<<"Display Base";
    }
    virtual void show()
    {
    cout<<"Show Base";
    } };
    class Derived : public Base
    { public: void
    display()
    { cout<<"Display
    Derived";
    }
    void show()
    { cout<<"show
    derived"; }
    };

void main()
{
    Base b;
    Derived d; Base
    *ptr;
    cout<<"ptr points to Base"; ptr=&b;
    ptr->display(); //calls Base
    ptr->show(); //calls Base
    cout<<"ptr points to derived";
    ptr=&d;
    ptr->display(); //calls Base ptr-
    >show(); //class Derived
}

```

Output:

```

ptr point s to Base
Display Base
Show Base

```

```

ptr points to Derived

```

```

Display Base
Show Derived

```

When ptr is made to point to the object d, the statement ptr->display(); calls only the function associated with the Base i.e.. Base::display()

where as the statement

```
ptr->show();
```

calls the derived version of show(). This is because the function display() has not been made virtual in the Base class.

Rules For Virtual Functions:

When virtual functions are created for implementing late binding, observe some basic rules that satisfy the compiler requirements.

1. The virtual functions must be members of some class.
2. They cannot be static members.
3. They are accessed by using object pointers.
4. A virtual function can be a friend of another class.
5. A virtual function in a base class must be defined, even though it may not be used.
6. The prototypes of the base class version of a virtual function and all the derived class versions must be identical. C++ considers them as overloaded functions, and the virtual function mechanism is ignored.
7. We cannot have virtual constructors, but we can have virtual destructors.
8. While a base pointer points to any type of the derived object, the reverse is not true. i.e. we cannot use a pointer to a derived class to access an object of the base class type.
9. When a base pointer points to a derived class, incrementing or decrementing it will not make it to point to the next object of the derived class. It is incremented or decremented only relative to its base type. Therefore we should not use this method to move the pointer to the next object.
10. If a virtual function is defined in the base class, it need not be necessarily redefined in the derived class. In such cases, calls will invoke the base function.

OVERLOADING

OPERATOR OVERLOADING

C++ has the ability to provide the operators with as special meaning for a data type. The mechanism of giving such special meanings to an operator is known as operator overloading. We can overload all the operators except the following:

Class member access operator (“.” And “.*”) Scope resolution operator “::”
Size operator (sizeof)
Conditional operator

To define an additional task to an operator, specify what it means in relation to the class to which the operator is applied. This is done with the help of a special function, **called operator function.**

The process of overloading involves following steps:

1. Create a class that defines the data type that is to be used in the overloading operation.
2. Declare the operator function **operator op()** in the public part of the class. It may be a member function or a friend function.
3. Here op is the operator to be overloaded.
4. Define the operator function to implement the required operations.

General Form:-

```
return-type classname :: operator op(arglist)
{
    Function body
}
```

Ex:

```
complex complex::operator+(complex c)
{
    complex t;
    t.real=real+c.real;
    t.img=img+c.img;
    return t;
}
```

Concept of Operator Overloading

One of the unique features of C++ is Operator Overloading. Applying overloading to operators means, same operator in responding different manner. For example operator + can be used as concatenate operator as well as additional operator.

That is 2+3 means 5 (addition), where as
"2"+"3" means 23 (concatenation).

Performing many actions with a single operator is operator overloading. We can assign a user defined function to an operator. We can change function of an operator, but it is not recommended to change the actual functions of operator. We can't create new operators using this operatorloading.

Operator overloading concept can be applied in following two major areas (Benefits)

1. Extension of usage of operators
2. Data conversions

Rules to be followed for operator overloading:-

1. Only existing operators can be overloaded.
2. Overloaded operators must have at least one operand that is of user defined operators
3. We cannot change basic meaning of an operator.
4. Overloaded operator must follow minimum characteristics that of original operator
5. When using binary operator overloading through member function, the left hand operand must be an object of relevant class

The number of arguments in the overloaded operator's arguments list depends

1. Operator function must be either member function or friend function.

2. If operator function is a friend function then it will have one argument for unary operator & two arguments for binary operator
3. If operator function is a member function then it will have Zero argument for unary operator & one arguments for binary operator

Unary Operator Overloading

An unary operator means, an operator which works on single operand. For example, ++ is an unary operator, it takes single operand (c++). So, when overloading an unary operator, it takes no argument (because object itself is considered as argument).

Syntax for Unary Operator (Inside a class)

```
return-type operator operatorsymbol()
{
//body of the function
}
```

Ex:

```
void operator-()
{ real=-real;
  img=-img;
}
```

Syntax for Unary Operator (Outside a class)

```
return-
type classname::operator operatorsymbol( )
{
    //body of the function
}
```

Example 1:-

```
void operator++()
{
    counter++;
}
```

Example 2:-

```
void complex::operator-()
{
    real=-real; img=-img;
}
```

The following simple program explains the concept of unary overloading.

```
#include < iostream.h >
#include < conio.h >
// Program Operator
Overloading class fact
{ int a;

public:
    fact ()
```

```

        {
            a=0;
        }
        fact (int i)
        { a=i;
        } fact operator!()
        { int
        f=1,i;
        fact t;
        for (i=1;i<=a;i++)
        { f=f*i;
        }
        t.a=f; return
        t;
    }
    void display()
    {
        cout<<"The factorial "<< a;
    }
}; void
main()
{ int
x;

    cout<<"enter a number";
    cin>>x; fact
s(x),p;
p=!s;
p.display();
}

```

Output for the above program:

Enter a number 5

The factorial of a given number 120

Explanation:

We have taken „!“ as operator to overload. Here class name is fact. Constructor without parameters to take initially value of „x“ as 0. Constructor with parameter to take the value of „x“. We have create two objects one for doing the factorial and the other for return the factorial. Here number of parameter for an overloaded function is 0. Factorial is unary operator because it operates on one dataitem. operator overloading find the factorial of the object. The display function for printing the result.

Overloading Unary Operator -

Example 1: -

Write a program to overload unary operator –

```

#include<iostream>
using namespace
std; class complex {

```

```

float real,img;
public:
    complex();
    complex(float x, float y);
    void display();
void operator-();
};
complex::complex(
) { real=0;img=0; }
complex::complex(float x, float y)
{ real=x; img=y; } void
complex::display() {
int imag=img;

if(img<0)
{ imag=-img; cout<<real<<" -
i"<<img<<endl;
} else cout<<real<<"
+i"<<img<<endl;
} void
complex::operator-() {
real=-real; img=-img; }
int main() { complex
c(1,-2); c.display();
cout<<"After Unary - operation\n";
-c;
c.display();
}

```

Example 2: -

```

#include<iostream.h> using
namespace std; class space {
int x,y,z; public: void
getdata(int a,int b,int c);
void display();

void operator -();
};
void space :: getdata(int a,int b,int c)
{
x=a;
y=b;
z=c;
}
void space :: display()
{
cout<<"x="<<x<<endl;
cout<<"y="<<y<<endl;
cout<<"z="<<z<<endl;
}

```

```

}
void space :: operator-()
{ x=-x;
y=-y; z=-
z; } int
main() {
space s;
s.getdata(1 0,-20,30);
s.display();
-s;
cout<<"after negation\n";
s.display();
}

```

Output:

```

      x=10
y=-20 z=30
after
negation x=-
10 y=20
z= -30

```

It is possible to overload a unary minus operator using a friend function as follows:

friend void operator-(space &s);

Example 3: -

Unary minus operator using a friend function

```

#include<iostream.h>
>
#include<iostream.h>
> using namespace
std; class space { int
x,y,z; public:
void getdata(int a,int b,int c);
void display(); friend void
operator-(space &s);
};
void space :: getdata(int a,int b,int c)
{
x=a;
y=b;
z=c;
}

void space :: display()
{
cout<<x<<" "<<y<<" "<<z<<endl;
}
void operator-(space &s)
{
s.x=-s.x;

```

```

s.y=-s.y;
s.z=-s.z;
}

int main()
{
space S;
S.getdata(10,-20,30);
S.display(); -
S;
cout<<"after negation\n";
S.display();
}

```

Output:

```

10    -20 30
after negation
-10 20-30

```

Binary Operator Overloading

An binary operator means, an operator which works on two operands. For example, + is an binary operator, it takes single operand (c+d). So, when overloading an binary operator, it takes one argument (one is object itself and other one is passed argument).

Syntax for Binary Operator (Inside a class)

```

return-type operator operatorsymbol(argument)
{
//body of the function
}

```

Syntax for Binary Operator definition (Outside a class)

```

return-type classname::operator operatorsymbol(argument)
{
//body of the function
}

```

Example complex

```

operator+(complex s)
{ complex
t;
t.real=real+s.real;
t.img=img+s.img;
return t;
}

```

The following program explains binary operator overloading:

```

#include <iostream.h>
> #include <conio.h>
class sum { int a;

```

```

public:
    sum()
    { a=0; }
    sum(int i)
    { a=i; }
    sum operator+(sum p1)
    { sum
      t;

      t.a=a+p1.a;
      return t;
    }
void main ()
{ cout<<"Enter Two Numbers:" int
  a,b;
  cin>>a>>b;
  sum x(a),y(b),z;
  z.display(); z=x+y; cout<<"after
applying operator \n"; z.display(); getch(); }

```

Output for the above program:

Enter two numbers 5 6

After applying operator

The sum of two numbers 11

Explanation: The class name is „sum“. We have create three objects two for to do the sum and the other for returning the sum. „+“ is a binary operator operates on members of two objects and returns the result which is member of a object.here number of parameters are 1. The sum is displayed in display function.

Write a program to over load arithmetic operators on complex numbers using member function

```

#include<iostream.h>
class complex
{ float real,img;
public:
    complex(){ }
    complex(float x, float y)
    { real=x;
      img=y
      ;
    }
    complex operator+(complex c) void
    display();
};
complex complex::operator+(complex c)
{ complex temp;
  temp.real=real+c.real
; temp.img=img+c.img;
  return temp;
}
void complex::display()
{ int
imag=img;
  If(img<0)
  { imag=-imag;

```



```

        cout<<real<<"-i"<<img;
    } else cout<<real<<" +i"<<img;
} int
main()
{
complex c1,c2,c3;
c1=complex(2.5,3.5);
c2=complex(1.6,2.7); c3=c1+c2;
    c3.display(); return
    0;
}

```

Overloading Binary Operators Using Friends

1. Replace the member function declaration by the friend function declaration in the class friend complex operator+(complex, complex) 2. Redefine the operator function as follows:

```

complex operator+(complex a, complex b)
{ return complex((a.x+b.x),(a.y+b.y));
}

```

Write a program to over load arithmetic operators on complex numbers using friend function #include<iostream.h> class complex

```

{ float real,img;
public:
    complex(){ }
    complex(float x, float y)
    { real=x;
      img=y;
    }
    friend complex operator+(complex); void
    display();
};
complex operator+(complex c1, complex c2)
{ complex temp;
temp.real=c1.real+c2.real;
    temp.img=c1.img+c2.img;
;
    return temp;
}
void complex::display()
{
    If(img<0)
    { img=-img;
      cout<<real<<"-i"<<img;
    } else cout<<real<<" +i"<<img;
} int
main() {
complex c1,c2,c3;
    c1=complex(2.5,3.5);
    c2=complex(1.6,2.7);
    c3=c1+c2;
}

```

```
c3.display();  
return 0;  
}
```